



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК4 Программное обеспечение ЭВМ, информационные технологии

ЛАБОРАТОРНАЯ РАБОТА №4

«Освещение в OpenGL»

по дисциплине: «Компьютерная графика»

Выполнил: студент группы ИУК4-43Б

(Подпись)

Афанасьева Ю.Р.

(И.О. Фамилия)

Проверил:

(Подпись)

Тихонович Е.О.

(И.О. Фамилия)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Цель: формирование практических навыков по работе с освещением средствами OpenGL, а также созданию настройки свойств материалов объектов и применения эффектов тени и тумана для большей реалистичности изображений.

Задачи: знать что такое затенение и особенности его применения к объектам различной геометрии; уметь работать с различными типами освещения и согласовывать свет с характеристиками материала; уметь устанавливать характеристики источников освещения, а также создавать блики и прожектора; понимать принципы формирования тени объекта. Уметь создавать тень на различных поверхностях; уметь реализовывать эффект отражения; понимать принципы формирования тумана и знать основные характеристики; понимать принципы сглаживания и уметь задавать необходимые в контексте каждой задачи параметры для выполнения эффекта сглаживания.

Вариант 3

Задание: воспроизвести результаты, представленные в теоретическом обзоре, применить различные виды освещения моделей, согласно варианту, полученному у преподавателя.

- 1) Выполнить затенение согласно варианту.
- 2) Продемонстрировать на 6 примерах различные варианты работы функций [glLightModelfv](#) и `glColorMaterial`. Модели выбрать согласно варианту (Примечание: для создания эффекта реалистичности изображения рекомендуется получить координаты полигонов объектов из сторонних приложений.)
- 3) На основе объекта из задания 2 продемонстрировать установку источников света согласно варианту (Примечание: для тех кто претендует на оценку «отлично» предусмотреть включение-выключение источника света через меню.)
- 4) На основе объекта из задания 2 продемонстрировать создание бликов на объекте.
- 5) На основе объекта из задания 2 создать прожектор согласно варианту
- 6) На основе объекта из задания 2 создать тень согласно варианту
- 7) На основе листинга 6 (7) согласно варианту из лабораторной работы №3 продемонстрировать эффект отражения.
- 8) На основе листинга 6 и 7 согласно варианту из лабораторной работы №3 создать туман с разными параметрами. Должен посредством меню меняться цвет, режим и уравнение тумана.
- 9) На основе листингов 2, 3, 4 и 5 лабораторной работы №2 продемонстрировать работу механизма сглаживания.

Этапы выполнения работы:

1. Выполнено плавное затенение пирамиды с квадратным основанием с использованием модели GL_SMOOTH и интерполяции цветов вершин.
2. Построена модель легкового автомобиля и подготовлены шесть примеров работы функций glLightModelfv и glColorMaterial с переключением через контекстное меню.
3. Для модели автомобиля установлены четыре источника света одного цвета: два источника расположены рядом, два других — в противоположных октантах.
4. На поверхности автомобиля созданы зеркальные блики посредством настройки компонента GL_SPECULAR и коэффициента блеска GL_SHININESS.
5. Созданы три прожектора в разных октантах с углами конусов 20, 45 и 60 градусов и различными оттенками серого цвета.
6. Для трёх источников света рассчитаны матрицы теневой проекции и построены три тени автомобиля на плоской земле с произвольным углом наклона.
7. На основе сцены варианта 3 лабораторной работы №3, содержащей 40 цветных сфер, тор и вращающуюся сферу, реализован эффект отражения от полупрозрачной поверхности.
8. Для сцены варианта 3 лабораторной работы №3 реализован туман с возможностью изменения его состояния, цвета и уравнения GL_LINEAR, GL_EXP или GL_EXP2 через контекстное меню.
9. На основе листингов 2–5 лабораторной работы №2 продемонстрировано сглаживание точек и линий с возможностью выбора примитива и включения или выключения сглаживания через контекстное меню.

Задание 1: Выполнить затенение пирамиды с квадратным основанием.

Листинг 1

```
#include <glut.h>

// Параметры поворота
static GLfloat xRot = 25.0f;
static GLfloat yRot = -35.0f;

// Вызывается для рисования сцены
void RenderScene(void)
{
    // Очистка окна текущим цветом очистки
    // Очистка буфера глубины
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Восстанавливаем нормальное состояние рисования
    glShadeModel(GL_SMOOTH);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);

    // Боковые грани пирамиды с квадратным основанием
    glBegin(GL_TRIANGLES);
        // Передняя грань
        glColor3ub((GLubyte)255, (GLubyte)0, (GLubyte)0);
        glVertex3f(0.0f, 180.0f, 0.0f);
        glColor3ub((GLubyte)0, (GLubyte)255, (GLubyte)0);
        glVertex3f(-140.0f, -100.0f, 140.0f);
        glColor3ub((GLubyte)0, (GLubyte)0, (GLubyte)255);
        glVertex3f(140.0f, -100.0f, 140.0f);

        // Правая грань
        glColor3ub((GLubyte)255, (GLubyte)0, (GLubyte)0);
        glVertex3f(0.0f, 180.0f, 0.0f);
        glColor3ub((GLubyte)0, (GLubyte)0, (GLubyte)255);
        glVertex3f(140.0f, -100.0f, 140.0f);
        glColor3ub((GLubyte)255, (GLubyte)255, (GLubyte)0);
        glVertex3f(140.0f, -100.0f, -140.0f);

        // Задняя грань
        glColor3ub((GLubyte)255, (GLubyte)0, (GLubyte)0);
        glVertex3f(0.0f, 180.0f, 0.0f);
        glColor3ub((GLubyte)255, (GLubyte)255, (GLubyte)0);
        glVertex3f(140.0f, -100.0f, -140.0f);
        glColor3ub((GLubyte)255, (GLubyte)0, (GLubyte)255);
        glVertex3f(-140.0f, -100.0f, -140.0f);
    glEnd();
}
```

```

        // Левая грань
        glColor3ub((GLubyte)255, (GLubyte)0, (GLubyte)0);
        glVertex3f(0.0f, 180.0f, 0.0f);
        glColor3ub((GLubyte)255, (GLubyte)0, (GLubyte)255);
        glVertex3f(-140.0f, -100.0f, -140.0f);
        glColor3ub((GLubyte)0, (GLubyte)255, (GLubyte)0);
        glVertex3f(-140.0f, -100.0f, 140.0f);
    glEnd();

    // Квадратное основание
    glBegin(GL_QUADS);
        glColor3ub((GLubyte)0, (GLubyte)255, (GLubyte)0);
        glVertex3f(-140.0f, -100.0f, 140.0f);
        glColor3ub((GLubyte)255, (GLubyte)0, (GLubyte)255);
        glVertex3f(-140.0f, -100.0f, -140.0f);
        glColor3ub((GLubyte)255, (GLubyte)255, (GLubyte)0);
        glVertex3f(140.0f, -100.0f, -140.0f);
        glColor3ub((GLubyte)0, (GLubyte)0, (GLubyte)255);
        glVertex3f(140.0f, -100.0f, 140.0f);
    glEnd();

    // Очищает очередь текущих команд
    glutSwapBuffers();
}

// Эта функция выполняет необходимую инициализацию в контексте
// визуализации
void SetupRC()
{
    // Черный фон
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glEnable(GL_DEPTH_TEST);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;

    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;

    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;

    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 356.0f)
        xRot = 0.0f;
}

```

```

    if (xRot < -1.0f)
        xRot = 355.0f;

    if (yRot > 356.0f)
        yRot = 0.0f;

    if (yRot < -1.0f)
        yRot = 355.0f;

    // Обновляется окно
    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat windowHeight, windowWidth;

    // Предотвращает деление на ноль, когда окно слишком маленькое
    if (h == 0)
        h = 1;

    // Размер поля просмотра устанавливается равным размеру окна
    glViewport(0, 0, w, h);

    // Обновление системы координат перед модификациями
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    if (w <= h)
    {
        windowHeight = 250.0f * h / w;
        windowWidth = 250.0f;
    }
    else
    {
        windowWidth = 250.0f * w / h;
        windowHeight = 250.0f;
    }

    // Установить объем отсечения
    glOrtho(-windowWidth, windowWidth,
            -windowHeight, windowHeight,
            -500.0f, 500.0f);

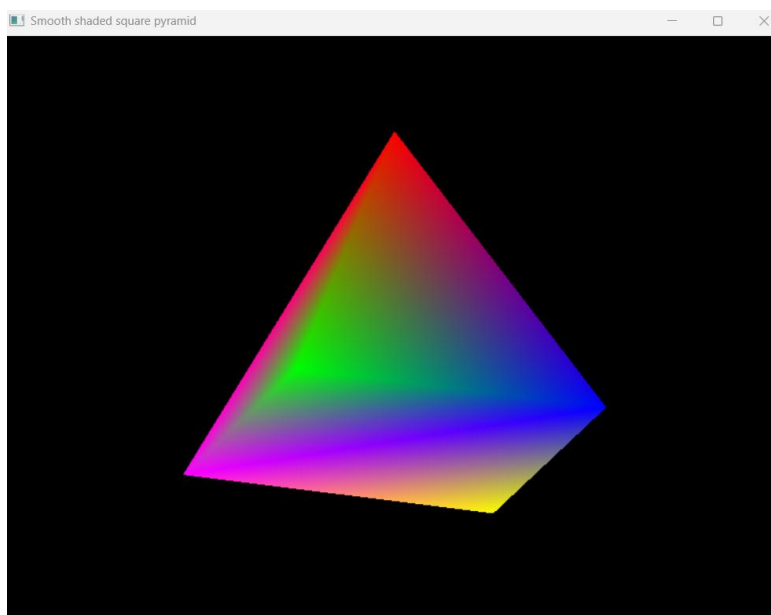
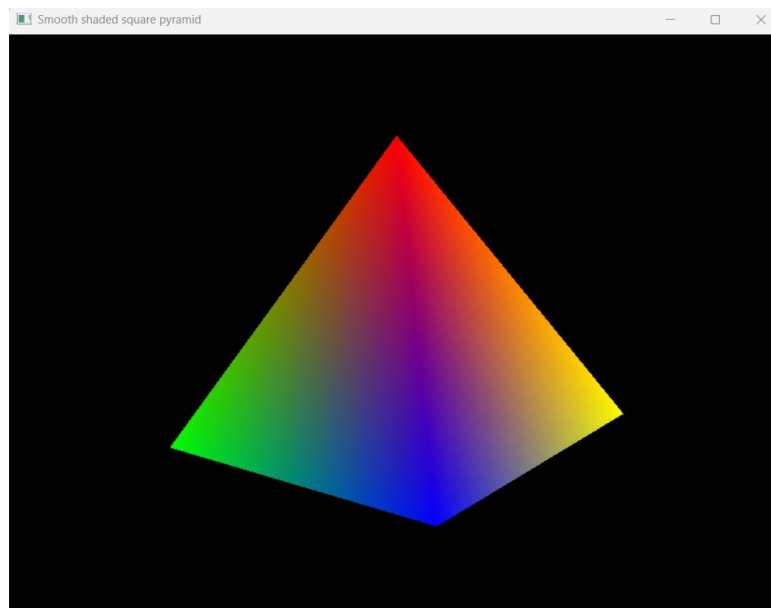
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);

```

```
glutInitWindowSize(800, 600);  
glutCreateWindow("Smooth shaded square pyramid");  
glutReshapeFunc(ChangeSize);  
glutSpecialFunc(SpecialKeys);  
glutDisplayFunc(RenderScene);  
SetupRC();  
glutMainLoop();  
return 0;  
}
```

Результаты работы программы 1



Задание 2: Продемонстрировать на 6 примерах различные варианты работы функций `glLightModelfv` и `glColorMaterial`. Модели выбрать согласно варианту (Примечание: для создания эффекта реалистичности изображения рекомендуется получить координаты полигонов объектов из сторонних приложений.) Легковой автомобиль.

Листинг 2

```
#include <glut.h>

// Параметры поворота
static GLfloat xRot = 20.0f;
static GLfloat yRot = -25.0f;
static int currentExample = 1;

void DrawBox(GLfloat x1, GLfloat y1, GLfloat z1,
             GLfloat x2, GLfloat y2, GLfloat z2)
{
    glBegin(GL_QUADS);
        glNormal3f(0.0f, 0.0f, 1.0f);
        glVertex3f(x1, y1, z2);
        glVertex3f(x2, y1, z2);
        glVertex3f(x2, y2, z2);
        glVertex3f(x1, y2, z2);

        glNormal3f(0.0f, 0.0f, -1.0f);
        glVertex3f(x2, y1, z1);
        glVertex3f(x1, y1, z1);
        glVertex3f(x1, y2, z1);
        glVertex3f(x2, y2, z1);

        glNormal3f(-1.0f, 0.0f, 0.0f);
        glVertex3f(x1, y1, z1);
        glVertex3f(x1, y1, z2);
        glVertex3f(x1, y2, z2);
        glVertex3f(x1, y2, z1);

        glNormal3f(1.0f, 0.0f, 0.0f);
        glVertex3f(x2, y1, z2);
        glVertex3f(x2, y1, z1);
        glVertex3f(x2, y2, z1);
        glVertex3f(x2, y2, z2);

        glNormal3f(0.0f, 1.0f, 0.0f);
        glVertex3f(x1, y2, z2);
        glVertex3f(x2, y2, z2);
        glVertex3f(x2, y2, z1);
        glVertex3f(x1, y2, z1);

        glNormal3f(0.0f, -1.0f, 0.0f);
        glVertex3f(x1, y1, z1);
```



```

        glVertex3f(x2, y1, z1);
        glVertex3f(x2, y1, z2);
        glVertex3f(x1, y1, z2);
    glEnd();
}

void DrawWheel(GLfloat x, GLfloat z)
{
    glPushMatrix();
    glTranslatef(x, -9.0f, z);
    glScalef(1.0f, 1.0f, 0.45f);
    glutSolidSphere(7.0, 20, 12);
    glPopMatrix();
}

void DrawCar()
{
    // Устанавливается цвет материала
    glColor3ub(190, 30, 30);
    DrawBox(-35.0f, -8.0f, -14.0f, 35.0f, 7.0f, 14.0f);

    glColor3ub(210, 45, 45);
    DrawBox(-17.0f, 7.0f, -12.0f, 18.0f, 23.0f, 12.0f);

    glColor3ub(40, 70, 100);
    DrawBox(-14.0f, 10.0f, -12.5f, 15.0f, 20.0f, 12.5f);

    glColor3ub(35, 35, 35);
    DrawWheel(-23.0f, 14.0f);
    DrawWheel(23.0f, 14.0f);
    DrawWheel(-23.0f, -14.0f);
    DrawWheel(23.0f, -14.0f);
}

void DrawExample(GLfloat x, GLfloat y, const GLfloat light[4], GLenum
materialMode)
{
    const GLfloat material[] = {0.2f, 0.2f, 0.2f, 1.0f};

    glLightModelfv(GL_LIGHT_MODEL_AMBIENT, light);

    glDisable(GL_COLOR_MATERIAL);
    glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, material);
    glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, material);

    // Активизируется согласование цветов
    glEnable(GL_COLOR_MATERIAL);
    glColorMaterial(GL_FRONT_AND_BACK, materialMode);

    glPushMatrix();
    glTranslatef(x, y, 0.0f);

```

```

    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    glScalef(1.5f, 1.5f, 1.5f);
    DrawCar();
    glPopMatrix();
}

// Вызывается для рисования сцены
void RenderScene(void)
{
    const GLfloat brightLight[] = {1.0f, 1.0f, 1.0f, 1.0f};
    const GLfloat halfLight[] = {0.5f, 0.5f, 0.5f, 1.0f};
    const GLenum materialModes[] = {
        GL_AMBIENT,
        GL_DIFFUSE,
        GL_AMBIENT_AND_DIFFUSE
    };

    // Очищаем окно текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    if (currentExample <= 3)
        DrawExample(0.0f, 0.0f, brightLight,
                    materialModes[currentExample - 1]);
    else
        DrawExample(0.0f, 0.0f, halfLight,
                    materialModes[currentExample - 4]);

    // Отобразить результаты
    glutSwapBuffers();
}

void ProcessMenu(int value)
{
    currentExample = value;
    glutPostRedisplay();
}

// Функция выполняет необходимую инициализацию
// в контексте визуализации
void SetupRC()
{
    glEnable(GL_DEPTH_TEST);

    // Освещение материала
    glEnable(GL_LIGHTING);

    // Светло-синий цвет
    glClearColor(0.0f, 0.0f, 0.5f, 1.0f);
}

```

```

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;

    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;

    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;

    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 356.0f)
        xRot = 0.0f;

    if (xRot < -1.0f)
        xRot = 355.0f;

    if (yRot > 356.0f)
        yRot = 0.0f;

    if (yRot < -1.0f)
        yRot = 355.0f;

    // Обновляется окно
    glutPostRedisplay();
}

```

```

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    // Предотвращение деления на ноль
    if (h == 0)
        h = 1;

    // Устанавливает поле просмотра по размерам окна
    glViewport(0, 0, w, h);

    // Обновляет стек матрицы проектирования
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Устанавливает объем отсечения с помощью отсекающих
    // плоскостей (левая, правая, нижняя, верхняя,
    // ближняя, дальняя)
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w,
                nRange * h / w, -nRange, nRange);
}

```

```

else
    glOrtho(-nRange * w / h, nRange * w / h,
            -nRange, nRange, -nRange, nRange);

glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Light Model and Color Material Cars");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();

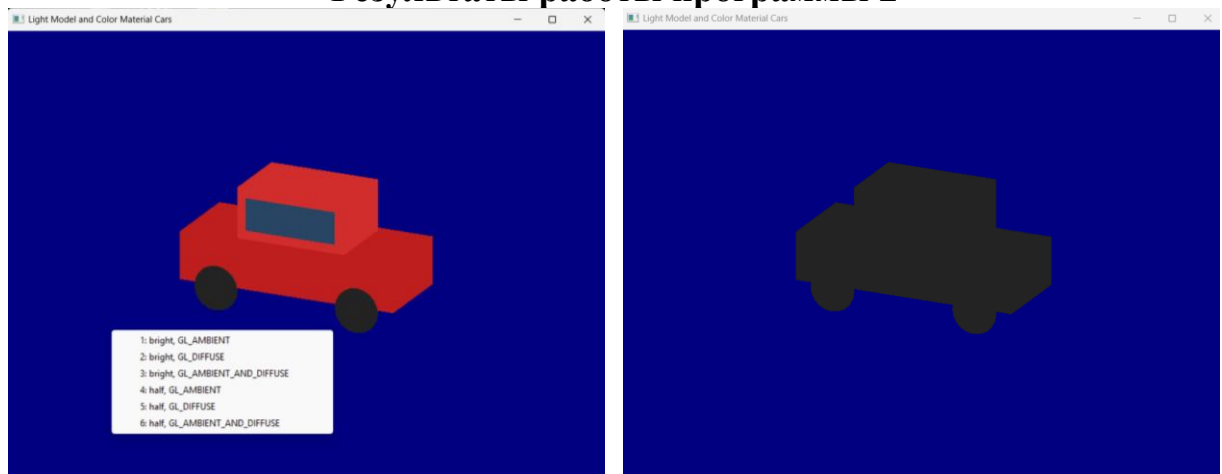
    glutCreateMenu(ProcessMenu);
    glutAddMenuEntry("1: bright, GL_AMBIENT", 1);
    glutAddMenuEntry("2: bright, GL_DIFFUSE", 2);
    glutAddMenuEntry("3: bright, GL_AMBIENT_AND_DIFFUSE", 3);
    glutAddMenuEntry("4: half, GL_AMBIENT", 4);
    glutAddMenuEntry("5: half, GL_DIFFUSE", 5);
    glutAddMenuEntry("6: half, GL_AMBIENT_AND_DIFFUSE", 6);
    glutAttachMenu(GLUT_RIGHT_BUTTON);

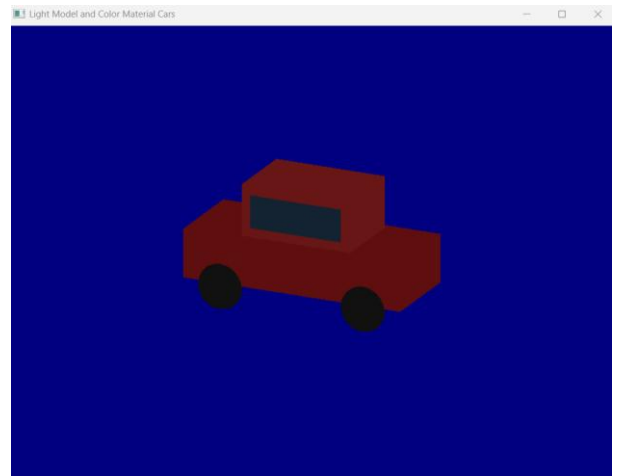
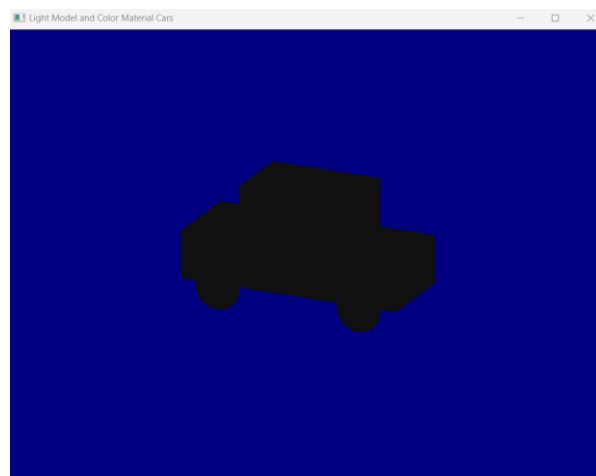
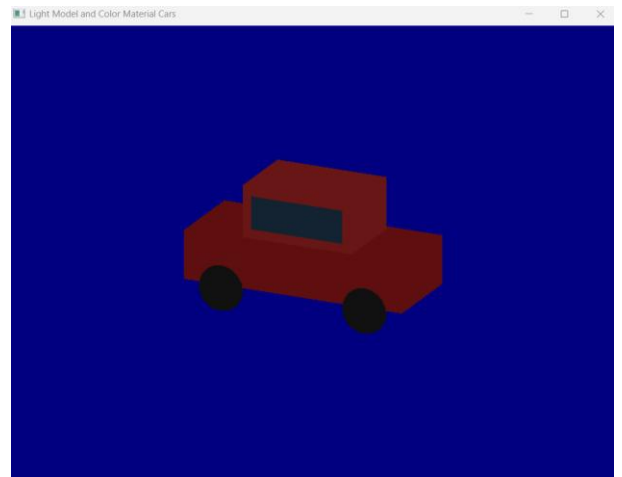
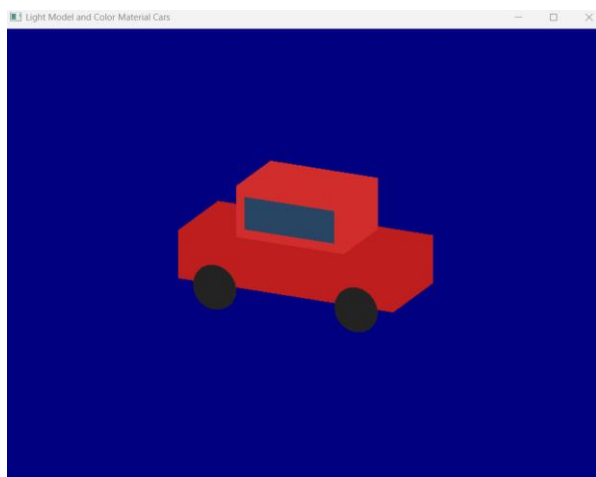
    glutMainLoop();

    return 0;
}

```

Результаты работы программы 2





Задание 3: Установить четыре источника света (два источника света находятся близко друг к другу, другие два в противоположных октантах) одного цвета.

Листинг 3

```
#include <glut.h>

// Величина поворота
static GLfloat xRot = 20.0f;
static GLfloat yRot = -25.0f;

void DrawBox(GLfloat x1, GLfloat y1, GLfloat z1,
             GLfloat x2, GLfloat y2, GLfloat z2)
{
    glBegin(GL_QUADS);
        glNormal3f(0.0f, 0.0f, 1.0f);
        glVertex3f(x1, y1, z2);
        glVertex3f(x2, y1, z2);
        glVertex3f(x2, y2, z2);
        glVertex3f(x1, y2, z2);

        glNormal3f(0.0f, 0.0f, -1.0f);
        glVertex3f(x2, y1, z1);
        glVertex3f(x1, y1, z1);
        glVertex3f(x1, y2, z1);
        glVertex3f(x2, y2, z1);

        glNormal3f(-1.0f, 0.0f, 0.0f);
        glVertex3f(x1, y1, z1);
        glVertex3f(x1, y1, z2);
        glVertex3f(x1, y2, z2);
        glVertex3f(x1, y2, z1);

        glNormal3f(1.0f, 0.0f, 0.0f);
        glVertex3f(x2, y1, z2);
        glVertex3f(x2, y1, z1);
        glVertex3f(x2, y2, z1);
        glVertex3f(x2, y2, z2);

        glNormal3f(0.0f, 1.0f, 0.0f);
        glVertex3f(x1, y2, z2);
        glVertex3f(x2, y2, z2);
        glVertex3f(x2, y2, z1);
        glVertex3f(x1, y2, z1);

        glNormal3f(0.0f, -1.0f, 0.0f);
        glVertex3f(x1, y1, z1);
        glVertex3f(x2, y1, z1);
        glVertex3f(x2, y1, z2);
        glVertex3f(x1, y1, z2);
    }
}
```

```

        glEnd();
    }

void DrawWheel(GLfloat x, GLfloat z)
{
    glPushMatrix();
    glTranslatef(x, -9.0f, z);
    glScalef(1.0f, 1.0f, 0.45f);
    glutSolidSphere(7.0, 20, 12);
    glPopMatrix();
}

void DrawCar()
{
    // Устанавливается цвет материала
    glColor3ub(128, 128, 128);
    DrawBox(-35.0f, -8.0f, -14.0f, 35.0f, 7.0f, 14.0f);
    DrawBox(-17.0f, 7.0f, -12.0f, 18.0f, 23.0f, 12.0f);

    glColor3ub(70, 70, 70);
    DrawBox(-14.0f, 10.0f, -12.5f, 15.0f, 20.0f, 12.5f);

    glColor3ub(35, 35, 35);
    DrawWheel(-23.0f, 14.0f);
    DrawWheel(23.0f, 14.0f);
    DrawWheel(-23.0f, -14.0f);
    DrawWheel(23.0f, -14.0f);
}

// Вызывается для рисования сцены
void RenderScene(void)
{
    // Очистка окна текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Записываем состояние матрицы и выполняем поворот
    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    glScalef(1.5f, 1.5f, 1.5f);
    DrawCar();
    glPopMatrix();

    // Отобразить результаты
    glutSwapBuffers();
}

// Эта функция выполняет необходимую инициализацию в контексте
// визуализации
void SetupRC()
{

```

```

// Коды и координаты источников света
GLfloat ambientLight[] = {0.075f, 0.075f, 0.075f, 1.0f};
GLfloat diffuseLight[] = {0.175f, 0.175f, 0.175f, 1.0f};

GLfloat lightPos0[] = {20.0f, 70.0f, 20.0f, 1.0f};
GLfloat lightPos1[] = {28.0f, 75.0f, 25.0f, 1.0f};
GLfloat lightPos2[] = {-80.0f, 50.0f, -80.0f, 1.0f};
GLfloat lightPos3[] = {80.0f, -50.0f, 80.0f, 1.0f};

glEnable(GL_DEPTH_TEST);

// Активизируется освещение
glEnable(GL_LIGHTING);

// Устанавливаются и активизируются источники света
glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight);
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuseLight);
glLightfv(GL_LIGHT0, GL_POSITION, lightPos0);
glEnable(GL_LIGHT0);

glLightfv(GL_LIGHT1, GL_AMBIENT, ambientLight);
glLightfv(GL_LIGHT1, GL_DIFFUSE, diffuseLight);
glLightfv(GL_LIGHT1, GL_POSITION, lightPos1);
glEnable(GL_LIGHT1);

glLightfv(GL_LIGHT2, GL_AMBIENT, ambientLight);
glLightfv(GL_LIGHT2, GL_DIFFUSE, diffuseLight);
glLightfv(GL_LIGHT2, GL_POSITION, lightPos2);
glEnable(GL_LIGHT2);

glLightfv(GL_LIGHT3, GL_AMBIENT, ambientLight);
glLightfv(GL_LIGHT3, GL_DIFFUSE, diffuseLight);
glLightfv(GL_LIGHT3, GL_POSITION, lightPos3);
glEnable(GL_LIGHT3);

// Активизирует согласование цветов
glEnable(GL_COLOR_MATERIAL);

// Свойства материалов соответствуют кодам glColor
glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE);

// Светло-голубой фон
glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
}

// Управление стрелками
void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
}

```



```

    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;

    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;

    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 356.0f)
        xRot = 0.0f;

    if (xRot < -1.0f)
        xRot = 355.0f;

    if (yRot > 356.0f)
        yRot = 0.0f;

    if (yRot < -1.0f)
        yRot = 355.0f;

    // Обновляется окно
    glutPostRedisplay();
}

// Обновляется проекция и положение источника света
void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    // Предотвращает деление на ноль
    if (h == 0)
        h = 1;

    // Устанавливает поле просмотра по размерам окна
    glViewport(0, 0, w, h);

    // Обновляет стек матрицы проектирования
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Устанавливает объем отсечения с помощью отсекающих
    // плоскостей (левая, правая, нижняя, верхняя,
    // ближняя, дальняя)
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w,
                nRange * h / w, -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h,
                -nRange, nRange, -nRange, nRange);
}

```

```

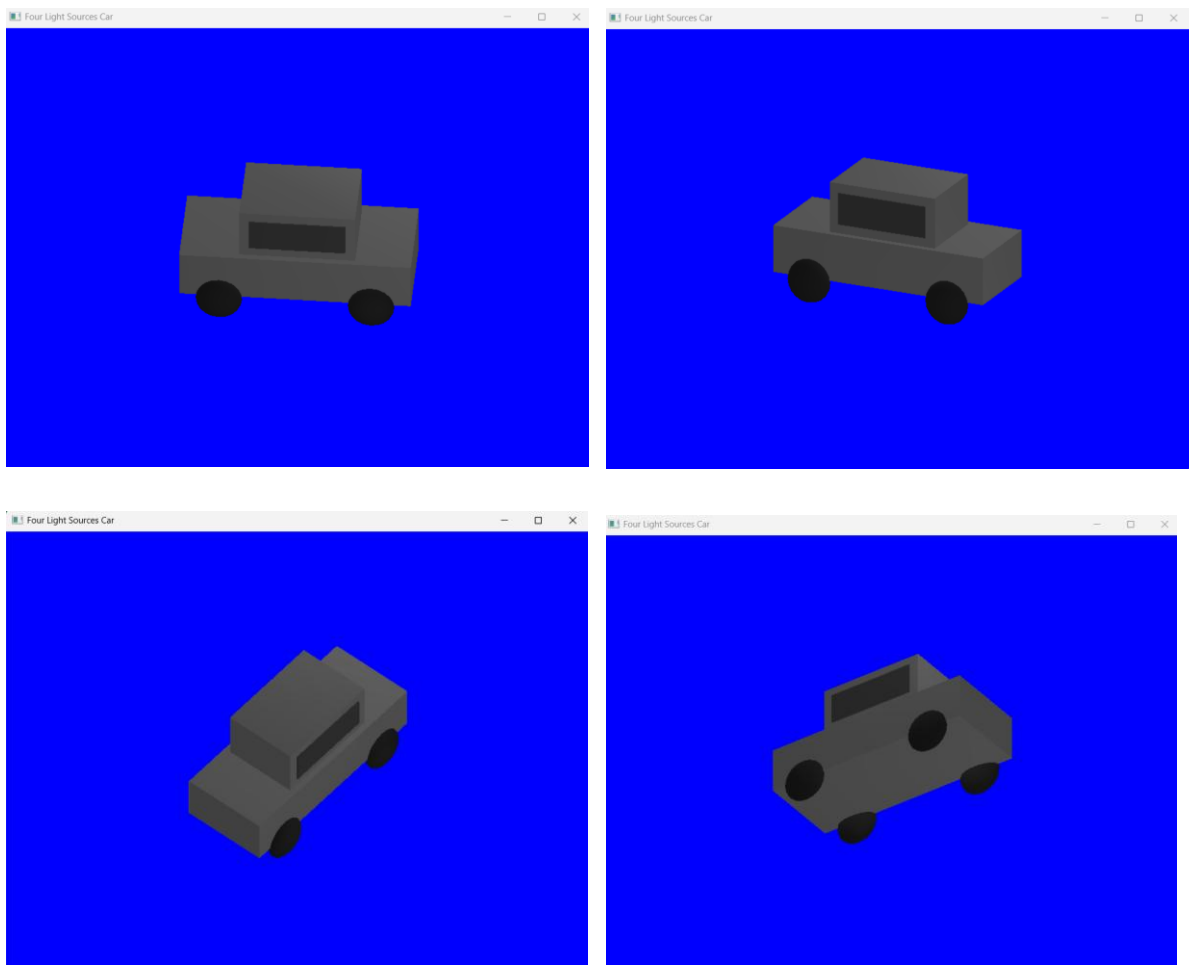
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Four Light Sources Car");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();

    return 0;
}

```

Результаты работы программы 3



Задание 4: На основе объекта из задания 2 продемонстрировать создание бликов на объекте.

Листинг 4

```
#include <glut.h>

// Величина поворота
static GLfloat xRot = 20.0f;
static GLfloat yRot = -25.0f;

void DrawBox(GLfloat x1, GLfloat y1, GLfloat z1,
             GLfloat x2, GLfloat y2, GLfloat z2)
{
    glBegin(GL_QUADS);
        glNormal3f(0.0f, 0.0f, 1.0f);
        glVertex3f(x1, y1, z2);
        glVertex3f(x2, y1, z2);
        glVertex3f(x2, y2, z2);
        glVertex3f(x1, y2, z2);

        glNormal3f(0.0f, 0.0f, -1.0f);
        glVertex3f(x2, y1, z1);
        glVertex3f(x1, y1, z1);
        glVertex3f(x1, y2, z1);
        glVertex3f(x2, y2, z1);

        glNormal3f(-1.0f, 0.0f, 0.0f);
        glVertex3f(x1, y1, z1);
        glVertex3f(x1, y1, z2);
        glVertex3f(x1, y2, z2);
        glVertex3f(x1, y2, z1);

        glNormal3f(1.0f, 0.0f, 0.0f);
        glVertex3f(x2, y1, z2);
        glVertex3f(x2, y1, z1);
        glVertex3f(x2, y2, z1);
        glVertex3f(x2, y2, z2);

        glNormal3f(0.0f, 1.0f, 0.0f);
        glVertex3f(x1, y2, z2);
        glVertex3f(x2, y2, z2);
        glVertex3f(x2, y2, z1);
        glVertex3f(x1, y2, z1);

        glNormal3f(0.0f, -1.0f, 0.0f);
        glVertex3f(x1, y1, z1);
        glVertex3f(x2, y1, z1);
        glVertex3f(x2, y1, z2);
        glVertex3f(x1, y1, z2);
    glEnd();
}
```

```

}

void DrawWheel(GLfloat x, GLfloat z)
{
    glPushMatrix();
    glTranslatef(x, -9.0f, z);
    glScalef(1.0f, 1.0f, 0.45f);
    glutSolidSphere(7.0, 20, 12);
    glPopMatrix();
}

void DrawCar()
{
    // Устанавливается цвет материала
    glColor3ub(128, 128, 128);
    DrawBox(-35.0f, -8.0f, -14.0f, 35.0f, 7.0f, 14.0f);
    DrawBox(-17.0f, 7.0f, -12.0f, 18.0f, 23.0f, 12.0f);

    glColor3ub(70, 70, 70);
    DrawBox(-14.0f, 10.0f, -12.5f, 15.0f, 20.0f, 12.5f);

    glColor3ub(35, 35, 35);
    DrawWheel(-23.0f, 14.0f);
    DrawWheel(23.0f, 14.0f);
    DrawWheel(-23.0f, -14.0f);
    DrawWheel(23.0f, -14.0f);
}

// Вызывается для рисования сцены
void RenderScene(void)
{
    // Очистка окна текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Записываем состояние матрицы и выполняем поворот
    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    glScalef(1.5f, 1.5f, 1.5f);
    DrawCar();
    glPopMatrix();
    // Отобразить результаты
    glutSwapBuffers();
}

// Эта функция выполняет необходимую инициализацию в контексте
// визуализации
void SetupRC()
{
    // Координаты и освещение
    GLfloat ambientLight[] = {0.3f, 0.3f, 0.3f, 1.0f};

```

```

GLfloat diffuseLight[] = {0.7f, 0.7f, 0.7f, 1.0f};
GLfloat specular[] = {1.0f, 1.0f, 1.0f, 1.0f};
GLfloat specref[] = {1.0f, 1.0f, 1.0f, 1.0f};
GLfloat lightPos[] = {-50.0f, 50.0f, 100.0f, 1.0f};

glEnable(GL_DEPTH_TEST);

// Активируется освещение
glEnable(GL_LIGHTING);

// Устанавливается и активируется источник света 0
glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight);
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuseLight);
glLightfv(GL_LIGHT0, GL_SPECULAR, specular);
glLightfv(GL_LIGHT0, GL_POSITION, lightPos);
glEnable(GL_LIGHT0);

// Активирует согласование цветов
glEnable(GL_COLOR_MATERIAL);

// Свойства материалов соответствуют кодам glColor
glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE);

// Все материалы в дальнейшем имеют полное зеркальное отражение
// с высоким блеском
glMaterialfv(GL_FRONT, GL_SPECULAR, specref);
glMateriali(GL_FRONT, GL_SHININESS, 128);

// Светло-голубой фон
glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
}

// Управление стрелками
void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;

    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;

    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;

    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 356.0f)
        xRot = 0.0f;

    if (xRot < -1.0f)

```

```

        xRot = 355.0f;

    if (yRot > 356.0f)
        yRot = 0.0f;

    if (yRot < -1.0f)
        yRot = 355.0f;

    // Обновляется окно
    glutPostRedisplay();
}

// Обновляется проекция и положение источника света
void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    // Предотвращает деление на нуль
    if (h == 0)
        h = 1;

    // Размер поля просмотра устанавливается равным размеру окна
    glViewport(0, 0, w, h);

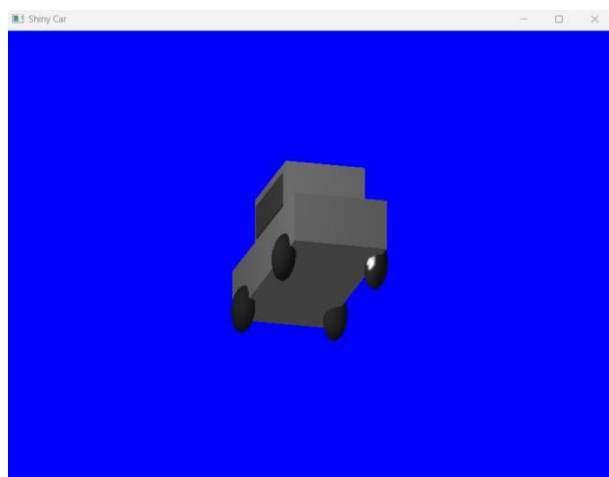
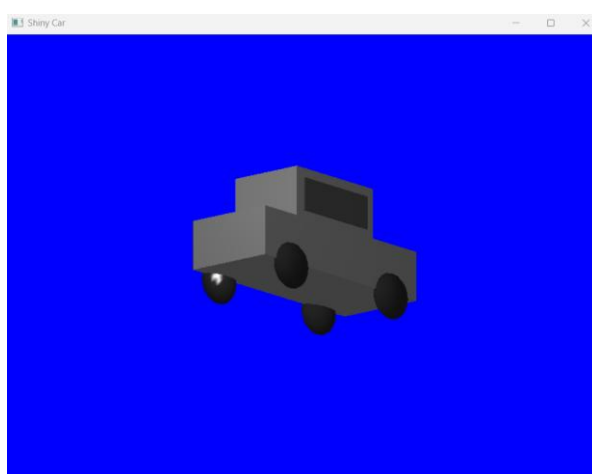
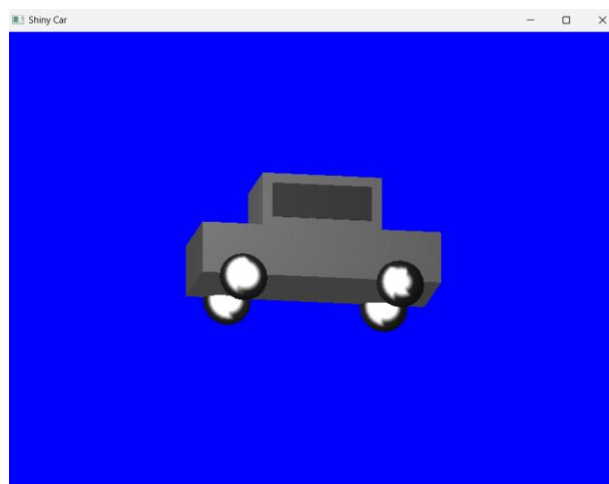
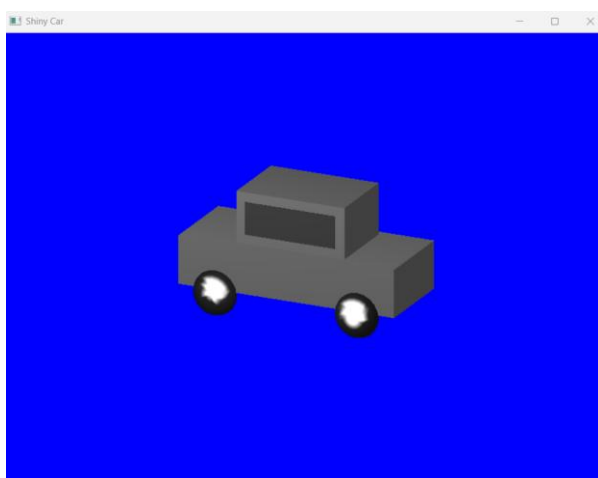
    // Обновляется система координат
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w,
                nRange * h / w, -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h,
                -nRange, nRange, -nRange, nRange);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Shiny Car");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();

    return 0;
}

```

Результаты работы программы 4



Задание 5: Создать 3 прожектора в разных октантах по 20, 45 и 60 градусов. Цвет должен быть разный в тонах серый.

Листинг 5

```
#include <glut.h>

// Величина поворота
static GLfloat xRot = 20.0f;
static GLfloat yRot = -25.0f;

void DrawBox(GLfloat x1, GLfloat y1, GLfloat z1,
             GLfloat x2, GLfloat y2, GLfloat z2)
{
    glBegin(GL_QUADS);
        glNormal3f(0.0f, 0.0f, 1.0f);
        glVertex3f(x1, y1, z2);
        glVertex3f(x2, y1, z2);
        glVertex3f(x2, y2, z2);
        glVertex3f(x1, y2, z2);

        glNormal3f(0.0f, 0.0f, -1.0f);
        glVertex3f(x2, y1, z1);
        glVertex3f(x1, y1, z1);
        glVertex3f(x1, y2, z1);
        glVertex3f(x2, y2, z1);

        glNormal3f(-1.0f, 0.0f, 0.0f);
        glVertex3f(x1, y1, z1);
        glVertex3f(x1, y1, z2);
        glVertex3f(x1, y2, z2);
        glVertex3f(x1, y2, z1);

        glNormal3f(1.0f, 0.0f, 0.0f);
        glVertex3f(x2, y1, z2);
        glVertex3f(x2, y1, z1);
        glVertex3f(x2, y2, z1);
        glVertex3f(x2, y2, z2);

        glNormal3f(0.0f, 1.0f, 0.0f);
        glVertex3f(x1, y2, z2);
        glVertex3f(x2, y2, z2);
        glVertex3f(x2, y2, z1);
        glVertex3f(x1, y2, z1);

        glNormal3f(0.0f, -1.0f, 0.0f);
        glVertex3f(x1, y1, z1);
        glVertex3f(x2, y1, z1);
        glVertex3f(x2, y1, z2);
        glVertex3f(x1, y1, z2);
    glEnd();
}
```



```

}

void DrawWheel(GLfloat x, GLfloat z)
{
    glPushMatrix();
    glTranslatef(x, -9.0f, z);
    glScalef(1.0f, 1.0f, 0.45f);
    glutSolidSphere(7.0, 20, 12);
    glPopMatrix();
}

void DrawCar()
{
    // Устанавливается цвет материала
    glColor3ub(128, 128, 128);
    DrawBox(-35.0f, -8.0f, -14.0f, 35.0f, 7.0f, 14.0f);
    DrawBox(-17.0f, 7.0f, -12.0f, 18.0f, 23.0f, 12.0f);

    glColor3ub(70, 70, 70);
    DrawBox(-14.0f, 10.0f, -12.5f, 15.0f, 20.0f, 12.5f);

    glColor3ub(35, 35, 35);
    DrawWheel(-23.0f, 14.0f);
    DrawWheel(23.0f, 14.0f);
    DrawWheel(-23.0f, -14.0f);
    DrawWheel(23.0f, -14.0f);
}

// Вызывается для рисования сцены
void RenderScene(void)
{
    // Очищаем окно текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Записываем состояние матрицы и выполняем поворот
    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    glScalef(1.5f, 1.5f, 1.5f);
    DrawCar();
    glPopMatrix();

    // Отображаются результаты
    glutSwapBuffers();
}

// Эта функция выполняет необходимую инициализацию
// в контексте визуализации
void SetupRC()
{
    // Коэффициенты освещения и координаты

```

```

GLfloat ambientLight[] = {0.15f, 0.15f, 0.15f, 1.0f};
GLfloat specular[] = {1.0f, 1.0f, 1.0f, 1.0f};
GLfloat specref[] = {1.0f, 1.0f, 1.0f, 1.0f};

GLfloat light0[] = {60.0f, 70.0f, 80.0f, 1.0f};
GLfloat light1[] = {-70.0f, 60.0f, -60.0f, 1.0f};
GLfloat light2[] = {50.0f, -70.0f, -80.0f, 1.0f};

GLfloat spotDir0[] = {-60.0f, -70.0f, -80.0f};
GLfloat spotDir1[] = {70.0f, -60.0f, 60.0f};
GLfloat spotDir2[] = {-50.0f, 70.0f, 80.0f};

// Разные оттенки серого
GLfloat gray0[] = {0.35f, 0.35f, 0.35f, 1.0f};
GLfloat gray1[] = {0.65f, 0.65f, 0.65f, 1.0f};
GLfloat gray2[] = {1.0f, 1.0f, 1.0f, 1.0f};

glEnable(GL_DEPTH_TEST);

// Активируется освещение
glEnable(GL_LIGHTING);

// Создается слабое рассеянное освещение, чтобы
// можно было видеть объекты
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, ambientLight);

// Устанавливается и активируется источник света 0
glLightfv(GL_LIGHT0, GL_DIFFUSE, gray0);
glLightfv(GL_LIGHT0, GL_SPECULAR, specular);
glLightfv(GL_LIGHT0, GL_POSITION, light0);
glLightfv(GL_LIGHT0, GL_SPOT_DIRECTION, spotDir0);
// Прожекторные эффекты
glLightf(GL_LIGHT0, GL_SPOT_CUTOFF, 20.0f);
glEnable(GL_LIGHT0);

// Устанавливается и активируется источник света 1
glLightfv(GL_LIGHT1, GL_DIFFUSE, gray1);
glLightfv(GL_LIGHT1, GL_SPECULAR, specular);
glLightfv(GL_LIGHT1, GL_POSITION, light1);
glLightfv(GL_LIGHT1, GL_SPOT_DIRECTION, spotDir1);
// Прожекторные эффекты
glLightf(GL_LIGHT1, GL_SPOT_CUTOFF, 45.0f);
glEnable(GL_LIGHT1);

// Устанавливается и активируется источник света 2
glLightfv(GL_LIGHT2, GL_DIFFUSE, gray2);
glLightfv(GL_LIGHT2, GL_SPECULAR, specular);
glLightfv(GL_LIGHT2, GL_POSITION, light2);
glLightfv(GL_LIGHT2, GL_SPOT_DIRECTION, spotDir2);
// Прожекторные эффекты
glLightf(GL_LIGHT2, GL_SPOT_CUTOFF, 60.0f);

```

```

glEnable(GL_LIGHT2);

// Активируем согласование цветов
glEnable(GL_COLOR_MATERIAL);

// Свойства материалов соответствуют кодам glColor
glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE);

// С этого момента все материалы получают способность
// отражать блики
glMaterialfv(GL_FRONT, GL_SPECULAR, specref);
glMateriali(GL_FRONT, GL_SHININESS, 128);

// Черный фон
glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;

    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;

    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;

    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 356.0f)
        xRot = 0.0f;

    if (xRot < -1.0f)
        xRot = 355.0f;

    if (yRot > 356.0f)
        yRot = 0.0f;

    if (yRot < -1.0f)
        yRot = 355.0f;

    // Перерисовка окна
    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

```

```

// Предотвращает деление на ноль
if (h == 0)
    h = 1;

// Размер поля просмотра устанавливается равным размеру окна
glViewport(0, 0, w, h);

// Обновляется система координат
glMatrixMode(GL_PROJECTION);
glLoadIdentity();

if (w <= h)
    glOrtho(-nRange, nRange, -nRange * h / w,
            nRange * h / w, -nRange, nRange);
else
    glOrtho(-nRange * w / h, nRange * w / h,
            -nRange, nRange, -nRange, nRange);

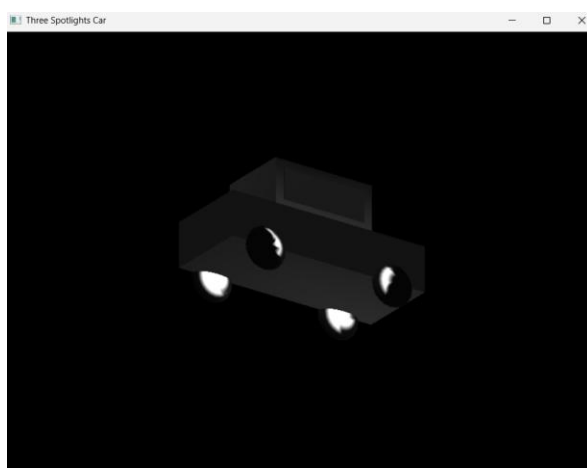
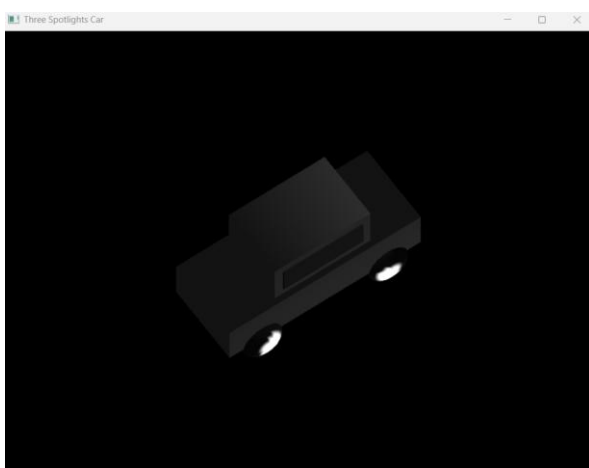
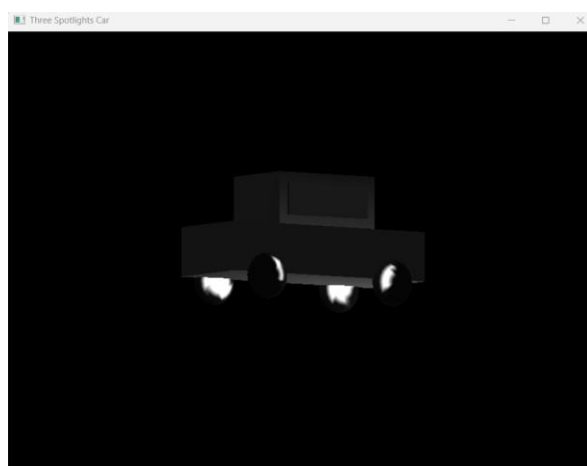
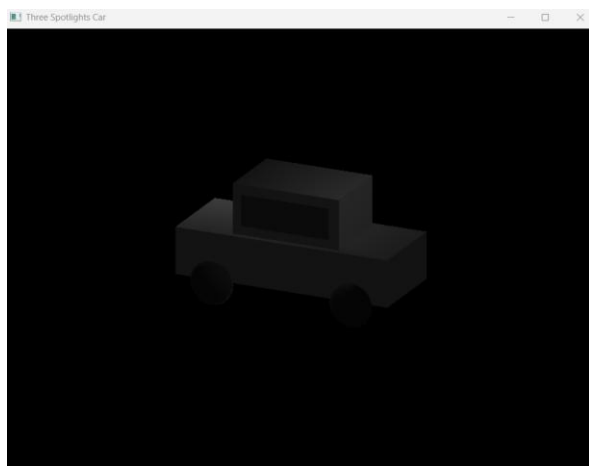
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Three Spotlights Car");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();

    return 0;
}

```

Результаты работы программы 5



Задание 6: На основе объекта из задания 2 создать тень. Создать три источника света. Земля плоская и имеет произвольный угол наклона.

Листинг 6

[illegible]

```

        vNormal[2] * vNormal[2]);

    vNormal[0] /= length;
    vNormal[1] /= length;
    vNormal[2] /= length;
}

// Три точки на плоскости расположены против часовой стрелки, вычисление
// нормали
void gltGetNormalVector(const GLTVector3 vP1,
                        const GLTVector3 vP2,
                        const GLTVector3 vP3,
                        GLTVector3 vNormal)
{
    GLTVector3 vV1, vV2;
    gltSubtractVectors(vP2, vP1, vV1);
    gltSubtractVectors(vP3, vP1, vV2);
    gltVectorCrossProduct(vV1, vV2, vNormal);
    gltNormalizeVector(vNormal);
}

// Полученные три коэффициента уравнения плоскости
// дают три точки на поверхности
void gltGetPlaneEquation(const GLTVector3 vPoint1,
                         const GLTVector3 vPoint2,
                         const GLTVector3 vPoint3,
                         GLTVector4 vPlane)
{
    gltGetNormalVector(vPoint1, vPoint2, vPoint3, vPlane);
    vPlane[3] = -(vPlane[0] * vPoint3[0] +
                  vPlane[1] * vPoint3[1] +
                  vPlane[2] * vPoint3[2]);
}

// Создание матрицы теневой проекции из коэффициентов
// уравнения плоскости и положения света
void gltMakeShadowMatrix(GLTVector3 vPoints[3],
                         GLTVector4 vLightPos,
                         GLTMatrix destMat)
{
    GLTVector4 vPlaneEquation;
    GLfloat dot;

    gltGetPlaneEquation(vPoints[0], vPoints[1], vPoints[2],
                        vPlaneEquation);

    // Скалярное произведение направляющего вектора
    // плоскости и положения источника света
    dot = vPlaneEquation[0] * vLightPos[0] +
          vPlaneEquation[1] * vLightPos[1] +
          vPlaneEquation[2] * vLightPos[2] +
          vPlaneEquation[3] * vLightPos[3];

```

```

// Выполняется проектирование
// Первый столбец
destMat[0] = dot - vLightPos[0] * vPlaneEquation[0];
destMat[4] = 0.0f - vLightPos[0] * vPlaneEquation[1];
destMat[8] = 0.0f - vLightPos[0] * vPlaneEquation[2];
destMat[12] = 0.0f - vLightPos[0] * vPlaneEquation[3];

// Второй столбец
destMat[1] = 0.0f - vLightPos[1] * vPlaneEquation[0];
destMat[5] = dot - vLightPos[1] * vPlaneEquation[1];
destMat[9] = 0.0f - vLightPos[1] * vPlaneEquation[2];
destMat[13] = 0.0f - vLightPos[1] * vPlaneEquation[3];

// Третий столбец
destMat[2] = 0.0f - vLightPos[2] * vPlaneEquation[0];
destMat[6] = 0.0f - vLightPos[2] * vPlaneEquation[1];
destMat[10] = dot - vLightPos[2] * vPlaneEquation[2];
destMat[14] = 0.0f - vLightPos[2] * vPlaneEquation[3];

// Четвертый столбец
destMat[3] = 0.0f - vLightPos[3] * vPlaneEquation[0];
destMat[7] = 0.0f - vLightPos[3] * vPlaneEquation[1];
destMat[11] = 0.0f - vLightPos[3] * vPlaneEquation[2];
destMat[15] = dot - vLightPos[3] * vPlaneEquation[3];
}

void DrawBox(GLfloat x1, GLfloat y1, GLfloat z1,
             GLfloat x2, GLfloat y2, GLfloat z2)
{
    glBegin(GL_QUADS);
        glNormal3f(0.0f, 0.0f, 1.0f);
        glVertex3f(x1, y1, z2); glVertex3f(x2, y1, z2);
        glVertex3f(x2, y2, z2); glVertex3f(x1, y2, z2);

        glNormal3f(0.0f, 0.0f, -1.0f);
        glVertex3f(x2, y1, z1); glVertex3f(x1, y1, z1);
        glVertex3f(x1, y2, z1); glVertex3f(x2, y2, z1);

        glNormal3f(-1.0f, 0.0f, 0.0f);
        glVertex3f(x1, y1, z1); glVertex3f(x1, y1, z2);
        glVertex3f(x1, y2, z2); glVertex3f(x1, y2, z1);

        glNormal3f(1.0f, 0.0f, 0.0f);
        glVertex3f(x2, y1, z2); glVertex3f(x2, y1, z1);
        glVertex3f(x2, y2, z1); glVertex3f(x2, y2, z2);

        glNormal3f(0.0f, 1.0f, 0.0f);
        glVertex3f(x1, y2, z2); glVertex3f(x2, y2, z2);
        glVertex3f(x2, y2, z1); glVertex3f(x1, y2, z1);
    glEnd();
}

```



```

        glNormal3f(0.0f, -1.0f, 0.0f);
        glVertex3f(x1, y1, z1); glVertex3f(x2, y1, z1);
        glVertex3f(x2, y1, z2); glVertex3f(x1, y1, z2);
    glEnd();
}

void DrawWheel(GLfloat x, GLfloat z)
{
    glPushMatrix();
    glTranslatef(x, -9.0f, z);
    glScalef(1.0f, 1.0f, 0.45f);
    glutSolidSphere(7.0, 20, 12);
    glPopMatrix();
}

// Функция, специально прорисовывающая автомобиль
void DrawCar(int nShadow)
{
    // Черный цвет используется только для тени
    if (nShadow == 0)
        glColor3ub(128, 128, 128);
    else
        glColor3ub(0, 0, 0);

    DrawBox(-35.0f, -8.0f, -14.0f, 35.0f, 7.0f, 14.0f);
    DrawBox(-17.0f, 7.0f, -12.0f, 18.0f, 23.0f, 12.0f);

    if (nShadow == 0)
        glColor3ub(70, 70, 70);
    DrawBox(-14.0f, 10.0f, -12.5f, 15.0f, 20.0f, 12.5f);

    if (nShadow == 0)
        glColor3ub(35, 35, 35);
    DrawWheel(-23.0f, 14.0f);
    DrawWheel(23.0f, 14.0f);
    DrawWheel(-23.0f, -14.0f);
    DrawWheel(23.0f, -14.0f);
}

void DrawShadow(GLTMatrix shadowMat)
{
    glPushMatrix();
    // Текущая матрица множится на матрицу проекции тени
    glMultMatrixf(shadowMat);
    glTranslatef(0.0f, 30.0f, 0.0f);
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    DrawCar(1);
    glPopMatrix();
}

```

```

// Вызывается окно для рисования
void RenderScene(void)
{
    // Очищает окно текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Рисуется земля
    glDisable(GL_LIGHTING);
    glColor3ub(0, 120, 0);
    glBegin(GL_QUADS);
        glVertex3f(-140.0f, -45.0f, -110.0f);
        glVertex3f(-140.0f, -25.0f, 110.0f);
        glVertex3f(140.0f, 15.0f, 110.0f);
        glVertex3f(140.0f, -5.0f, -110.0f);
    glEnd();

    // Рисуется автомобиль с новой ориентацией
    glEnable(GL_LIGHTING);
    glPushMatrix();
    glTranslatef(0.0f, 30.0f, 0.0f);
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    DrawCar(0);
    glPopMatrix();

    // Мы готовы рисовать тени и землю
    glDisable(GL_DEPTH_TEST);
    glDisable(GL_LIGHTING);
    DrawShadow(shadowMat0);
    DrawShadow(shadowMat1);
    DrawShadow(shadowMat2);

    // Рисуется источники света
    glColor3ub(255, 255, 0);
    glPushMatrix();
    glTranslatef(lightPos0[0], lightPos0[1], lightPos0[2]);
    glutSolidSphere(4.0f, 10, 10);
    glPopMatrix();
    glPushMatrix();
    glTranslatef(lightPos1[0], lightPos1[1], lightPos1[2]);
    glutSolidSphere(4.0f, 10, 10);
    glPopMatrix();
    glPushMatrix();
    glTranslatef(lightPos2[0], lightPos2[1], lightPos2[2]);
    glutSolidSphere(4.0f, 10, 10);
    glPopMatrix();

    // Восстанавливаются переменные состояния освещения
    glEnable(GL_DEPTH_TEST);

    // Отображаются результаты

```

```

        glutSwapBuffers();
    }

    // Эта функция выполняет необходимую инициализацию
    // в контексте визуализации
    void SetupRC()
    {
        GLfloat ambientLight[] = {0.1f, 0.1f, 0.1f, 1.0f};
        GLfloat diffuseLight[] = {0.35f, 0.35f, 0.35f, 1.0f};
        GLfloat specref[] = {1.0f, 1.0f, 1.0f, 1.0f};

        // Любые три точки на поверхности
        GLVector3 points[3] = {
            {-140.0f, -45.0f, -110.0f},
            {-140.0f, -25.0f, 110.0f},
            {140.0f, 15.0f, 110.0f}
        };

        glEnable(GL_DEPTH_TEST);
        glEnable(GL_LIGHTING);

        glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight);
        glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuseLight);
        glLightfv(GL_LIGHT0, GL_POSITION, lightPos0);
        glEnable(GL_LIGHT0);

        glLightfv(GL_LIGHT1, GL_AMBIENT, ambientLight);
        glLightfv(GL_LIGHT1, GL_DIFFUSE, diffuseLight);
        glLightfv(GL_LIGHT1, GL_POSITION, lightPos1);
        glEnable(GL_LIGHT1);

        glLightfv(GL_LIGHT2, GL_AMBIENT, ambientLight);
        glLightfv(GL_LIGHT2, GL_DIFFUSE, diffuseLight);
        glLightfv(GL_LIGHT2, GL_POSITION, lightPos2);
        glEnable(GL_LIGHT2);

        glEnable(GL_COLOR_MATERIAL);
        glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE);
        glMaterialfv(GL_FRONT, GL_SPECULAR, specref);
        glMateriali(GL_FRONT, GL_SHININESS, 128);

        // Расчет матриц проекции для рисования теней на земле
        gltMakeShadowMatrix(points, lightPos0, shadowMat0);
        gltMakeShadowMatrix(points, lightPos1, shadowMat1);
        gltMakeShadowMatrix(points, lightPos2, shadowMat2);

        // Светло-синий фон
        glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
    }

    void SpecialKeys(int key, int x, int y)

```

```

{
    if (key == GLUT_KEY_UP) xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN) xRot += 5.0f;
    if (key == GLUT_KEY_LEFT) yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT) yRot += 5.0f;

    if (xRot > 356.0f) xRot = 0.0f;
    if (xRot < -1.0f) xRot = 355.0f;
    if (yRot > 356.0f) yRot = 0.0f;
    if (yRot < -1.0f) yRot = 355.0f;

    // Перерисовка окна
    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 150.0f;

    // Предотвращает деление на ноль
    if (h == 0) h = 1;

    // Размер поля просмотра устанавливается равным размеру окна
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

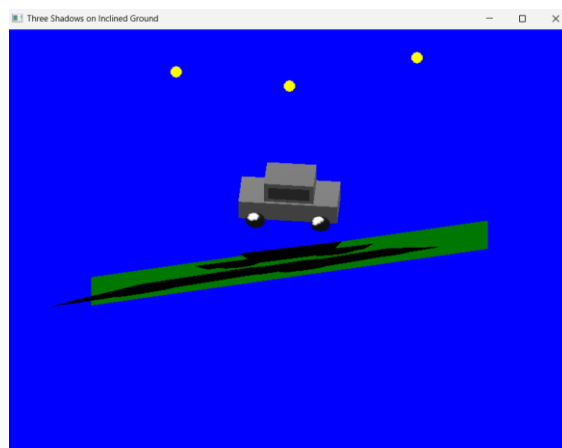
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w,
                nRange * h / w, -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h,
                -nRange, nRange, -nRange, nRange);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Three Shadows on Inclined Ground");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();
    return 0;
}

```

Результаты работы программы 6



Задание 7: На основе листинга 6 (7) согласно варианту из лабораторной работы №3 продемонстрировать эффект отражения.

Листинг 7

```
#include <glut.h>
#include <math.h>

#define GLT_PI 3.14159265358979323846
#define NUM_SPHERES 40

typedef GLfloat GLVector3[3];

struct SphereData
{
    GLfloat x;
    GLfloat y;
    GLfloat z;
    GLubyte red;
    GLubyte green;
    GLubyte blue;
};

SphereData spheres[NUM_SPHERES];
static GLfloat yRot = 0.0f;
static GLfloat viewRot = 20.0f;

// Масштабирование вектора по длине - создание единичного вектора
void gltNormalizeVector(GLVector3 vNormal)
{
    GLfloat length = (GLfloat)sqrt(vNormal[0] * vNormal[0] +
                                    vNormal[1] * vNormal[1] +
                                    vNormal[2] * vNormal[2]);

    vNormal[0] /= length;
    vNormal[1] /= length;
    vNormal[2] /= length;
}

void gltDrawTorus(GLfloat majorRadius, GLfloat minorRadius,
                  GLint numMajor, GLint numMinor)
{
    GLVector3 vNormal;
    double majorStep = 2.0 * GLT_PI / numMajor;
    double minorStep = 2.0 * GLT_PI / numMinor;

    for (int i = 0; i < numMajor; ++i)
    {
        double a0 = i * majorStep;
        double a1 = a0 + majorStep;
        GLfloat x0 = (GLfloat)cos(a0);
        GLfloat y0 = (GLfloat)sin(a0);
```

```

GLfloat x1 = (GLfloat)cos(a1);
GLfloat y1 = (GLfloat)sin(a1);

glBegin(GL_TRIANGLE_STRIP);
for (int j = 0; j <= numMinor; ++j)
{
    double b = j * minorStep;
    GLfloat c = (GLfloat)cos(b);
    GLfloat r = minorRadius * c + majorRadius;
    GLfloat z = minorRadius * (GLfloat)sin(b);

    vNormal[0] = x0 * c;
    vNormal[1] = y0 * c;
    vNormal[2] = z / minorRadius;
    gltNormalizeVector(vNormal);
    glNormal3fv(vNormal);
    glVertex3f(x0 * r, y0 * r, z);

    vNormal[0] = x1 * c;
    vNormal[1] = y1 * c;
    vNormal[2] = z / minorRadius;
    gltNormalizeVector(vNormal);
    glNormal3fv(vNormal);
    glVertex3f(x1 * r, y1 * r, z);
}
glEnd();
}
}

void DrawObjects()
{
    // Рисуются сферы на разном уровне по оси z
    for (int i = 0; i < NUM_SPHERES; ++i)
    {
        glPushMatrix();
        glTranslatef(spheres[i].x, spheres[i].y, spheres[i].z);
        glColor3ub(spheres[i].red, spheres[i].green, spheres[i].blue);
        glutSolidSphere(0.45f, 13, 13);
        glPopMatrix();
    }

    glPushMatrix();
    glTranslatef(0.0f, 1.1f, 0.0f);

    glPushMatrix();
    glRotatef(-yRot * 2.0f, 0.0f, 1.0f, 0.0f);
    glTranslatef(2.0f, 0.0f, 0.0f);
    glColor3ub(255, 220, 0);
    glutSolidSphere(0.45f, 13, 26);
    glPopMatrix();
}

```

```

    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    glColor3ub(220, 60, 40);
    glDrawTorus(1.2f, 0.45f, 40, 20);
    glPopMatrix();
}

// Вызывается для рисования сцены
void RenderScene(void)
{
    // Очищаем окно текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glRotatef(viewRot, 1.0f, 0.0f, 0.0f);

    // Рисуется отраженная сцена
    glPushMatrix();
    glScalef(1.0f, -1.0f, 1.0f);
    DrawObjects();
    glPopMatrix();

    // Активизируется смешение цветов
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

    // Рисуется полупрозрачная отражающая плоскость
    glDisable(GL_LIGHTING);
    glColor4f(0.15f, 0.25f, 0.35f, 0.65f);
    glBegin(GL_QUADS);
        glVertex3f(-11.0f, 0.0f, 11.0f);
        glVertex3f(11.0f, 0.0f, 11.0f);
        glVertex3f(11.0f, 0.0f, -11.0f);
        glVertex3f(-11.0f, 0.0f, -11.0f);
    glEnd();
    glDisable(GL_BLEND);

    // Рисуется исходная сцена
    glEnable(GL_LIGHTING);
    DrawObjects();

    // Переключение буферов
    glutSwapBuffers();
}

// Эта функция выполняет необходимую инициализацию в контексте
// визуализации
void SetupRC()
{
    GLfloat ambientLight[] = {0.3f, 0.3f, 0.3f, 1.0f};
    GLfloat diffuseLight[] = {0.7f, 0.7f, 0.7f, 1.0f};

```



```

GLfloat lightPos[] = {-5.0f, 10.0f, 5.0f, 1.0f};

// Сферы размещаются в уменьшенном пространстве
for (int i = 0; i < NUM_SPHERES; ++i)
{
    spheres[i].x = (GLfloat)((i * 37) % 180 - 90) * 0.1f;
    spheres[i].y = 0.45f;
    spheres[i].z = (GLfloat)((i * 67) % 180 - 90) * 0.1f;
    spheres[i].red = (GLubyte)(55 + (i * 47) % 200);
    spheres[i].green = (GLubyte)(55 + (i * 83) % 200);
    spheres[i].blue = (GLubyte)(55 + (i * 29) % 200);
}

glEnable(GL_DEPTH_TEST);
glEnable(GL_NORMALIZE);

// Активация освещения
glEnable(GL_LIGHTING);

// Устанавливается и активизируется источник света 0
glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight);
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuseLight);
glLightfv(GL_LIGHT0, GL_POSITION, lightPos);
glEnable(GL_LIGHT0);

// Активизирует согласование цветов
glEnable(GL_COLOR_MATERIAL);

// Свойства материалов соответствуют кодам glColor
glColorMaterial(GL_FRONT_AND_BACK, GL_AMBIENT_AND_DIFFUSE);

// Темно-синий фон
glClearColor(0.0f, 0.0f, 0.2f, 1.0f);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        viewRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        viewRot += 5.0f;

    // Обновление окна
    glutPostRedisplay();
}

// Вызывается библиотекой GLUT в холостом состоянии
void TimerFunction(int value)
{
    yRot += 1.0f;
    if (yRot > 360.0f)

```

```

        yRot = 0.0f;

        // Перерисовывает сцену с новыми координатами
        glutPostRedisplay();
        glutTimerFunc(33, TimerFunction, 1);
    }

void ChangeSize(int w, int h)
{
    GLfloat nRange = 13.0f;

    // Предотвращает деление на ноль, когда окно слишком маленькое
    if (h == 0)
        h = 1;

    glViewport(0, 0, w, h);

    // Система координат обновляется перед модификацией
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Устанавливается объём сечения
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w,
                nRange * h / w, -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h,
                -nRange, nRange, -nRange, nRange);

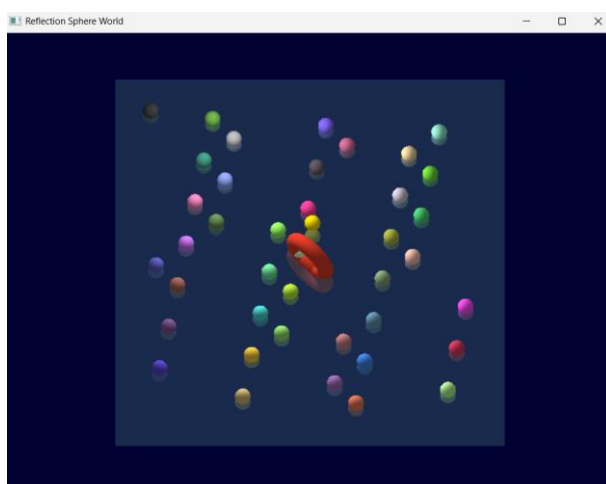
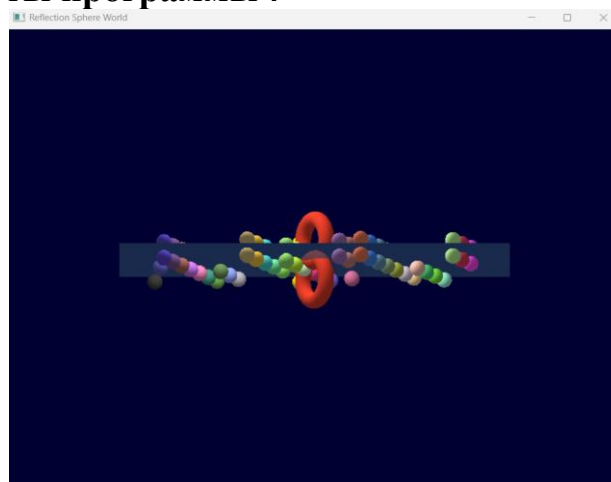
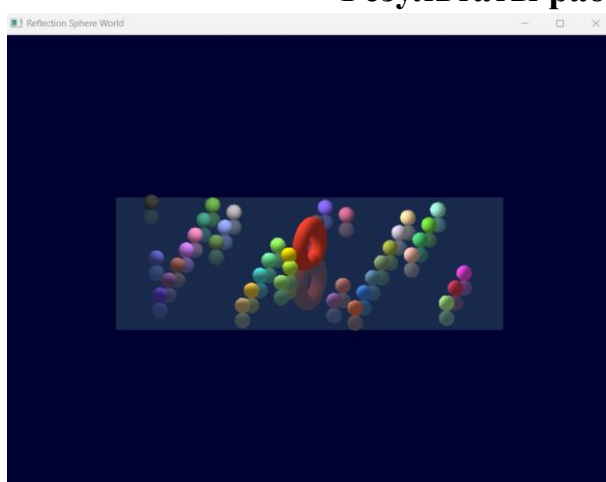
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Reflection Sphere World");
    glutReshapeFunc(ChangeSize);
    glutDisplayFunc(RenderScene);
    glutSpecialFunc(SpecialKeys);
    SetupRC();
    glutTimerFunc(33, TimerFunction, 1);
    glutMainLoop();

    return 0;
}

```

Результаты работы программы 7



Задание 8: На основе листинга 6 и 7 согласно варианту из лабораторной работы №3 создать туман с разными параметрами. Должен посредством меню меняться цвет, режим и уравнение тумана.

Листинг 8

```
#include <glut.h>
#include <math.h>

#define GLT_PI 3.14159265358979323846
#define NUM_SPHERES 40

typedef GLfloat GLTVector3[3];

struct SphereData
{
    GLfloat x;
    GLfloat y;
    GLfloat z;
    GLubyte red;
    GLubyte green;
    GLubyte blue;
};

SphereData spheres[NUM_SPHERES];
static GLfloat yRot = 0.0f;
static GLfloat viewRot = 20.0f;

GLfloat fogGray[] = {0.35f, 0.35f, 0.35f, 1.0f};
GLfloat fogBlue[] = {0.15f, 0.25f, 0.45f, 1.0f};
GLfloat fogWhite[] = {0.8f, 0.8f, 0.8f, 1.0f};

// Масштабирование вектора по длине - создание единичного вектора
void gltNormalizeVector(GLTVector3 vNormal)
{
    GLfloat length = (GLfloat)sqrt(vNormal[0] * vNormal[0] +
                                    vNormal[1] * vNormal[1] +
                                    vNormal[2] * vNormal[2]);

    vNormal[0] /= length;
    vNormal[1] /= length;
    vNormal[2] /= length;
}

void gltDrawTorus(GLfloat majorRadius, GLfloat minorRadius,
                  GLint numMajor, GLint numMinor)
{
    GLTVector3 vNormal;
    double majorStep = 2.0 * GLT_PI / numMajor;
    double minorStep = 2.0 * GLT_PI / numMinor;

    for (int i = 0; i < numMajor; ++i)
```

```

{
    double a0 = i * majorStep;
    double a1 = a0 + majorStep;
    GLfloat x0 = (GLfloat)cos(a0);
    GLfloat y0 = (GLfloat)sin(a0);
    GLfloat x1 = (GLfloat)cos(a1);
    GLfloat y1 = (GLfloat)sin(a1);

    glBegin(GL_TRIANGLE_STRIP);
    for (int j = 0; j <= numMinor; ++j)
    {
        double b = j * minorStep;
        GLfloat c = (GLfloat)cos(b);
        GLfloat r = minorRadius * c + majorRadius;
        GLfloat z = minorRadius * (GLfloat)sin(b);

        vNormal[0] = x0 * c;
        vNormal[1] = y0 * c;
        vNormal[2] = z / minorRadius;
        gltNormalizeVector(vNormal);
        glNormal3fv(vNormal);
        glVertex3f(x0 * r, y0 * r, z);

        vNormal[0] = x1 * c;
        vNormal[1] = y1 * c;
        vNormal[2] = z / minorRadius;
        gltNormalizeVector(vNormal);
        glNormal3fv(vNormal);
        glVertex3f(x1 * r, y1 * r, z);
    }
    glEnd();
}
}

// Рисуется земля с координатной сеткой
void DrawGround(void)
{
    GLfloat fExtent = 10.0f;
    GLfloat fStep = 1.0f;
    GLfloat y = 0.0f;

    glDisable(GL_LIGHTING);
    glColor3ub(100, 100, 100);
    glBegin(GL_LINES);
    for (GLfloat iLine = -fExtent; iLine <= fExtent; iLine += fStep)
    {
        glVertex3f(iLine, y, fExtent);
        glVertex3f(iLine, y, -fExtent);
        glVertex3f(fExtent, y, iLine);
        glVertex3f(-fExtent, y, iLine);
    }
}

```

```

    glEnd();
    glEnable(GL_LIGHTING);
}

void DrawWorld(void)
{
    DrawGround();

    // Рисуются сферы на разном уровне по оси z
    for (int i = 0; i < NUM_SPHERES; ++i)
    {
        glPushMatrix();
        glTranslatef(spheres[i].x, spheres[i].y, spheres[i].z);
        glColor3ub(spheres[i].red, spheres[i].green, spheres[i].blue);
        glutSolidSphere(0.45f, 13, 13);
        glPopMatrix();
    }

    glPushMatrix();
    glTranslatef(0.0f, 1.1f, 0.0f);

    glPushMatrix();
    glRotatef(-yRot * 2.0f, 0.0f, 1.0f, 0.0f);
    glTranslatef(2.0f, 0.0f, 0.0f);
    glColor3ub(255, 220, 0);
    glutSolidSphere(0.45f, 13, 26);
    glPopMatrix();

    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    glColor3ub(220, 60, 40);
    gltDrawTorus(1.2f, 0.45f, 40, 20);
    glPopMatrix();
}

// В ответ на выбор позиции меню устанавливаются
// соответствующие метки
void ProcessMenu(int value)
{
    switch (value)
    {
        case 1:
            glEnable(GL_FOG);
            break;
        case 2:
            glDisable(GL_FOG);
            break;
        case 3:
            glFogfv(GL_FOG_COLOR, fogGray);
            glClearColor(fogGray[0], fogGray[1], fogGray[2], fogGray[3]);
            break;
        case 4:

```

```

        glFogfv(GL_FOG_COLOR, fogBlue);
        glClearColor(fogBlue[0], fogBlue[1], fogBlue[2], fogBlue[3]);
        break;
    case 5:
        glFogfv(GL_FOG_COLOR, fogWhite);
        glClearColor(fogWhite[0], fogWhite[1], fogWhite[2], fogWhite[3]);
        break;
    case 6:
        glFogi(GL_FOG_MODE, GL_LINEAR);
        break;
    case 7:
        glFogi(GL_FOG_MODE, GL_EXP);
        break;
    case 8:
        glFogi(GL_FOG_MODE, GL_EXP2);
        break;
}

// Активизируем перерисовывание изображения
glutPostRedisplay();
}

// Вызывается для рисования сцены
void RenderScene(void)
{
    // Очищаем окно текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glRotatef(viewRot, 1.0f, 0.0f, 0.0f);
    DrawWorld();

    // Переключение буферов
    glutSwapBuffers();
}

// Эта функция выполняет необходимую инициализацию в контексте
// визуализации
void SetupRC()
{
    GLfloat ambientLight[] = {0.3f, 0.3f, 0.3f, 1.0f};
    GLfloat diffuseLight[] = {0.7f, 0.7f, 0.7f, 1.0f};
    GLfloat lightPos[] = {-5.0f, 10.0f, 5.0f, 1.0f};

    // Сферы размещаются в уменьшенном пространстве
    for (int i = 0; i < NUM_SPHERES; ++i)
    {
        spheres[i].x = (GLfloat)((i * 37) % 180 - 90) * 0.1f;
        spheres[i].y = 0.45f;
        spheres[i].z = (GLfloat)((i * 67) % 180 - 90) * 0.1f;
    }
}

```

```

        spheres[i].red = (GLubyte)(55 + (i * 47) % 200);
        spheres[i].green = (GLubyte)(55 + (i * 83) % 200);
        spheres[i].blue = (GLubyte)(55 + (i * 29) % 200);
    }

    glEnable(GL_DEPTH_TEST);
    glEnable(GL_NORMALIZE);
    glEnable(GL_LIGHTING);

    glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuseLight);
    glLightfv(GL_LIGHT0, GL_POSITION, lightPos);
    glEnable(GL_LIGHT0);

    glEnable(GL_COLOR_MATERIAL);
    glColorMaterial(GL_FRONT_AND_BACK, GL_AMBIENT_AND_DIFFUSE);

    // Установка параметров тумана
    glEnable(GL_FOG);
    glFogfv(GL_FOG_COLOR, fogGray);
    // Насколько далеко начинается туман
    glFogf(GL_FOG_START, 2.0f);
    // Насколько далеко заканчивается туман
    glFogf(GL_FOG_END, 12.0f);
    glFogf(GL_FOG_DENSITY, 0.18f);
    // Какое уравнение тумана используется
    glFogi(GL_FOG_MODE, GL_LINEAR);

    // Цвет тумана соответствует фону
    glClearColor(fogGray[0], fogGray[1], fogGray[2], fogGray[3]);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        viewRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        viewRot += 5.0f;

    // Обновление окна
    glutPostRedisplay();
}

// Вызывается библиотекой GLUT в холостом состоянии
void TimerFunction(int value)
{
    yRot += 1.0f;
    if (yRot > 360.0f)
        yRot = 0.0f;

    // Перерисовывает сцену с новыми координатами

```



```

        glutPostRedisplay();
        glutTimerFunc(33, TimerFunction, 1);
    }

void ChangeSize(int w, int h)
{
    GLfloat nRange = 13.0f;

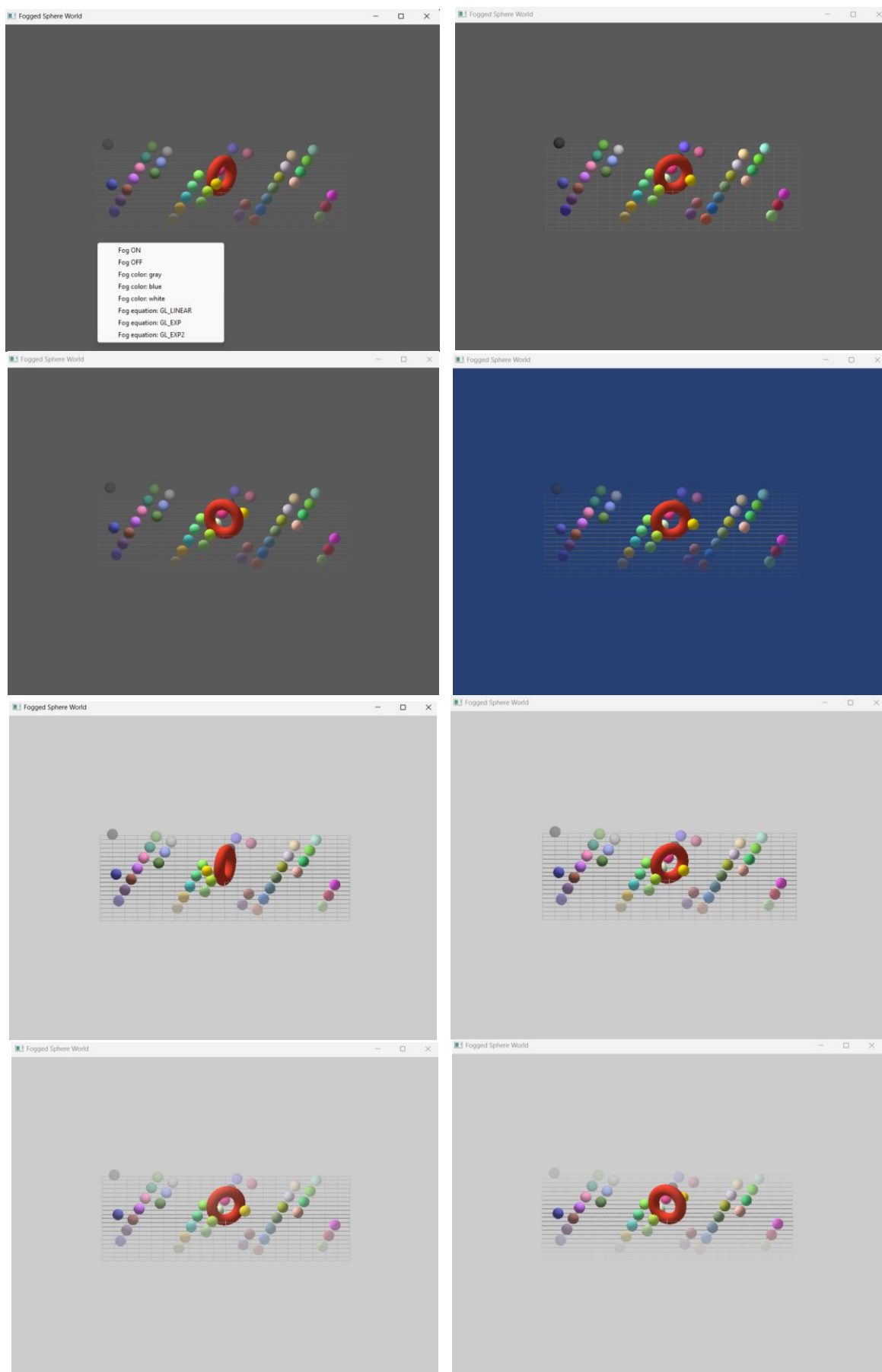
    // Предотвращает деление на ноль, когда окно слишком маленькое
    if (h == 0)
        h = 1;
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    // Устанавливается объём сечения
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w,
                nRange * h / w, -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h,
                -nRange, nRange, -nRange, nRange);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Fogged Sphere World");
    glutReshapeFunc(ChangeSize);
    glutDisplayFunc(RenderScene);
    glutSpecialFunc(SpecialKeys);
    glutTimerFunc(33, TimerFunction, 1);

    SetupRC();
    // Создаем меню
    glutCreateMenu(ProcessMenu);
    glutAddMenuEntry("Fog ON", 1);
    glutAddMenuEntry("Fog OFF", 2);
    glutAddMenuEntry("Fog color: gray", 3);
    glutAddMenuEntry("Fog color: blue", 4);
    glutAddMenuEntry("Fog color: white", 5);
    glutAddMenuEntry("Fog equation: GL_LINEAR", 6);
    glutAddMenuEntry("Fog equation: GL_EXP", 7);
    glutAddMenuEntry("Fog equation: GL_EXP2", 8);
    glutAttachMenu(GLUT_RIGHT_BUTTON);
    glutMainLoop();
    return 0;
}

```

Результаты работы программы 8



Задание 9: На основе листингов 2, 3, 4 и 5 лабораторной работы №2 продемонстрировать работу механизма сглаживания.

Листинг 9

```
#include <glut.h>
#include <math.h>

#define GL_PI 3.1415f

// Значения углов
static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;
static int currentListing = 2;

void DrawListing2(void)
{
    GLfloat x, y, z, angle;

    // Вызываем один раз для всех оставшихся точек
    glPointSize(5.0f);
    glBegin(GL_POINTS);

    z = -50.0f;
    for (angle = 0.0f; angle <= (2.0f * GL_PI) * 3.0f; angle += 0.1f)
    {
        x = 50.0f * (GLfloat)sin(angle);
        y = 50.0f * (GLfloat)cos(angle);

        // Задаем точку и немного смещаем значение z
        glVertex3f(x, y, z);
        z += 0.5f;
    }

    // Рисуем точки
    glEnd();
}

void DrawListing3(void)
{
    GLfloat x, y, z, angle;
    GLfloat sizes[2];
    GLfloat step;
    GLfloat curSize;

    // Получаем диапазон размеров поддерживаемых точек и размер шага
    glGetFloatv(GL_POINT_SIZE_RANGE, sizes);
    glGetFloatv(GL_POINT_SIZE_GRANULARITY, &step);

    // Задаем исходный размер точки
    curSize = sizes[0];
```

```

// Задаем начальную координату z
z = -50.0f;

// Циклический проход по окружности три раза
for (angle = 0.0f; angle <= (2.0f * GL_PI) * 3.0f; angle += 0.1f)
{
    // Расчет значений x и y точек окружности
    x = 50.0f * (GLfloat)sin(angle);
    y = 50.0f * (GLfloat)cos(angle);

    // Задаем размер точки перед указанием примитива
    glPointSize(curSize);

    // Рисуем точку
    glBegin(GL_POINTS);
    glVertex3f(x, y, z);
    glEnd();

    // Увеличиваем значение z и размер точки
    z += 0.5f;
    curSize += step;
}
}

void DrawListing4(void)
{
    GLfloat x, y, z, angle;

    glLineWidth(2.0f);
    glBegin(GL_LINES);

    // Все линии принадлежат плоскости xy
    z = 0.0f;
    for (angle = 0.0f; angle <= GL_PI; angle += GL_PI / 20.0f)
    {
        // Верхняя половина окружности
        x = 50.0f * (GLfloat)sin(angle);
        y = 50.0f * (GLfloat)cos(angle);
        glVertex3f(x, y, z);

        // Нижняя половина окружности
        x = 50.0f * (GLfloat)sin(angle + GL_PI);
        y = 50.0f * (GLfloat)cos(angle + GL_PI);
        glVertex3f(x, y, z);
    }

    // Рисуются точки
    glEnd();
}

```

```

void DrawListing5(void)
{
    GLfloat x, y, z, angle;

    glLineWidth(3.0f);

    // Вызывается один раз для всех точек
    glBegin(GL_LINE_STRIP);
    z = -50.0f;
    for (angle = 0.0f; angle <= (2.0f * GL_PI) * 3.0f; angle += 0.1f)
    {
        x = 50.0f * (GLfloat)sin(angle);
        y = 50.0f * (GLfloat)cos(angle);

        // Задаем точку и немного смещаем значение z
        glVertex3f(x, y, z);
        z += 0.5f;
    }

    // Рисует точки
    glEnd();
}

// В ответ на выбор позиции меню устанавливаются
// соответствующие метки
void ProcessMenu(int value)
{
    switch (value)
    {
        case 1:
            // Включается сглаживание и дается подсказка
            // обеспечить наивысшее качество
            glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
            glEnable(GL_BLEND);
            glEnable(GL_POINT_SMOOTH);
            glHint(GL_POINT_SMOOTH_HINT, GL_NICEST);
            glEnable(GL_LINE_SMOOTH);
            glHint(GL_LINE_SMOOTH_HINT, GL_NICEST);
            break;

        case 2:
            // Выключается смешивание и сглаживание
            glDisable(GL_BLEND);
            glDisable(GL_LINE_SMOOTH);
            glDisable(GL_POINT_SMOOTH);
            break;

        case 3:
            currentListing = 2;
            break;
    }
}

```

```

        case 4:
            currentListing = 3;
            break;

        case 5:
            currentListing = 4;
            break;

        case 6:
            currentListing = 5;
            break;
    }

    // Активизируем перерисовывание изображения
    glutPostRedisplay();
}

// Вызывается для рисования сцены
void RenderScene(void)
{
    // Окно очищается текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Записываем состояние матрицы и выполняем поворот
    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);

    if (currentListing == 2)
        DrawListing2();
    else if (currentListing == 3)
        DrawListing3();
    else if (currentListing == 4)
        DrawListing4();
    else
        DrawListing5();

    // Восстанавливаем преобразования
    glPopMatrix();

    // Очищаем стек команд рисования
    glutSwapBuffers();
}

// Функция выполняет необходимую инициализацию
// в контексте визуализации
void SetupRC(void)
{
    // Черный фон
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);

```

```

    // Цвет рисования выбирается зеленым
    glColor3f(0.0f, 1.0f, 0.0f);

    glEnable(GL_DEPTH_TEST);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 356.0f)
        xRot = 0.0f;
    if (xRot < -1.0f)
        xRot = 355.0f;
    if (yRot > 356.0f)
        yRot = 0.0f;
    if (yRot < -1.0f)
        yRot = 355.0f;

    // Обновляем окно
    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    // Предотвращает деление на ноль
    if (h == 0)
        h = 1;

    // Устанавливаем размеры поля просмотра равными размерам окна
    glViewport(0, 0, w, h);

    // Устанавливаем перспективную систему координат
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Устанавливает объем отсечения с помощью отсекающих
    // плоскостей (левая, правая, нижняя, верхняя, ближняя, дальняя)
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w,
                nRange * h / w, -nRange, nRange);
    else

```

```

        glOrtho(-nRange * w / h, nRange * w / h,
                -nRange, nRange, -nRange, nRange);

    // Обновляется стек матриц проекции модели
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

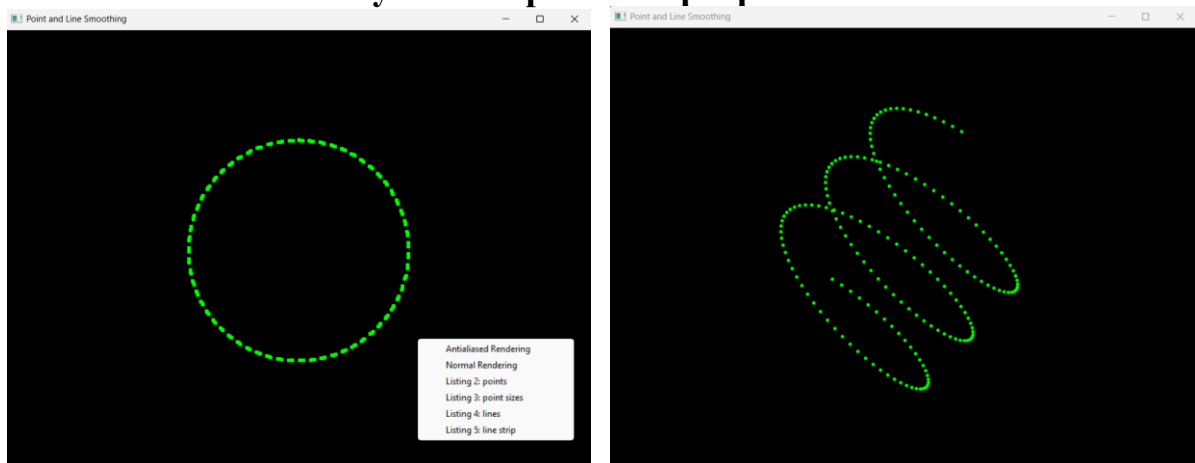
int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Point and Line Smoothing");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();

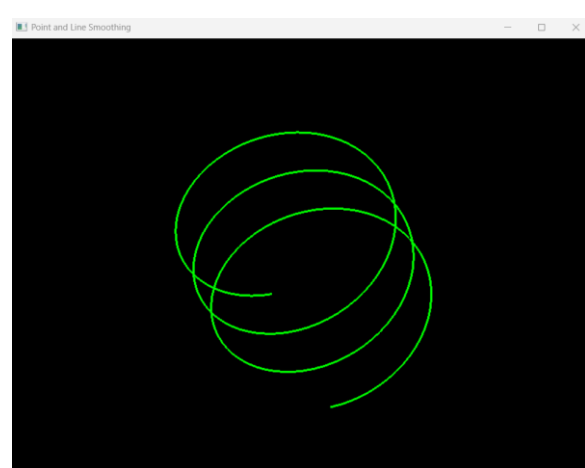
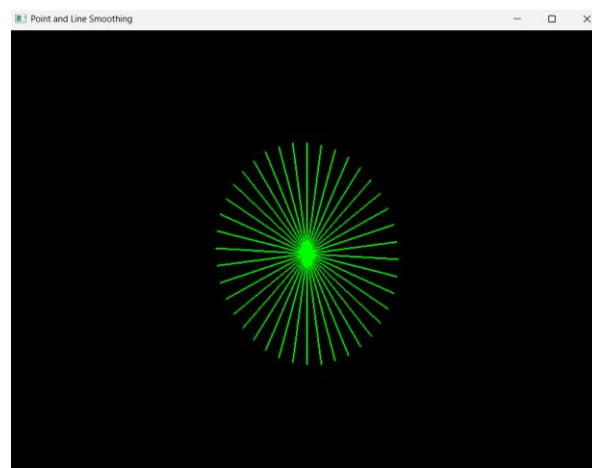
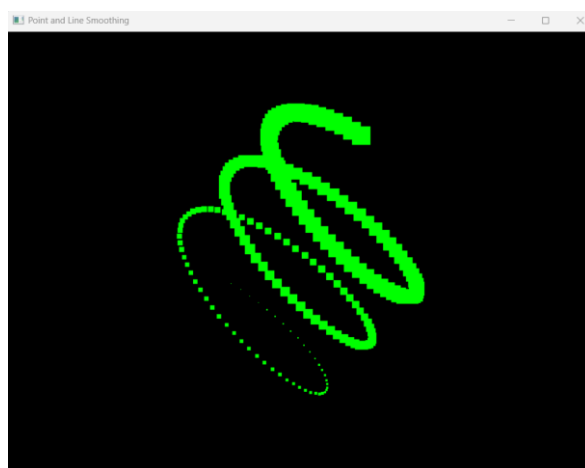
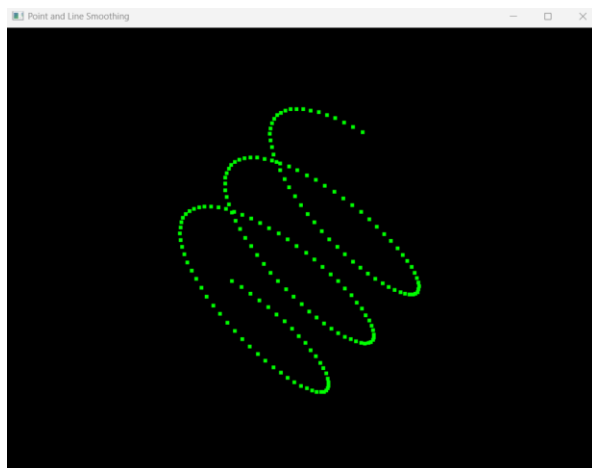
    // Создаем меню
    glutCreateMenu(ProcessMenu);
    glutAddMenuEntry("Antialiased Rendering", 1);
    glutAddMenuEntry("Normal Rendering", 2);
    glutAddMenuEntry("Listing 2: points", 3);
    glutAddMenuEntry("Listing 3: point sizes", 4);
    glutAddMenuEntry("Listing 4: lines", 5);
    glutAddMenuEntry("Listing 5: line strip", 6);
    glutAttachMenu(GLUT_RIGHT_BUTTON);

    glutMainLoop();
    return 0;
}

```

Результаты работы программы 9





Вывод: в ходе работы были получены практические навыки по работе с освещением средствами OpenGL, а также созданию настройки свойств материалов объектов и применения эффектов тени и тумана для большей реалистичности изображений.